

# Ejercicios Prácticos - Unidad 3, Sesión 2

## Acceso Programático a LLMs

### Configuración: Alternativa Gratuita con OpenRouter

¿No tienes API keys de pago? Puedes usar [OpenRouter](#) como proxy unificado para acceder a múltiples modelos de LLM, incluyendo **modelos gratuitos**. OpenRouter usa la misma librería [openai](#) de Python, solo cambia la configuración del cliente.

Pasos para usar OpenRouter:

1. Crea una cuenta en [openrouter.ai](#)
2. Obtén tu API key en [openrouter.ai/keys](#)
3. Añade a tu archivo `.env`:

```
OPENROUTER_API_KEY=sk-or-tu-clave-aqui
```

4. Configura el cliente así (en lugar del cliente estándar de OpenAI):

```
from openai import OpenAI
import os

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENROUTER_API_KEY"),
)
```

5. Usa modelos gratuitos. Consulta la lista actualizada en: <https://openrouter.ai/models?q=free&order=most-popular>

Algunos modelos gratuitos recomendados:

| Modelo                     | ID en OpenRouter                                 |
|----------------------------|--------------------------------------------------|
| Google Gemini 2.0 Flash    | <a href="#">google/gemini-2.0-flash-exp:free</a> |
| DeepSeek R1 (razonamiento) | <a href="#">deepseek/deepseek-r1-0528:free</a>   |
| Meta Llama 4 Scout         | <a href="#">meta-llama/llama-4-scout:free</a>    |
| Qwen3 30B                  | <a href="#">qwen/qwen3-30b-a3b:free</a>          |

**Nota:** En todos los ejercicios de esta sesión, cuando veas `client = OpenAI()` con modelo `gpt-4o-mini`, puedes sustituirlo por la configuración de OpenRouter con un modelo gratuito. Se indicará explícitamente en cada ejercicio.

## Ejercicio 1: Primera Llamada a la API

### Metadata

- **Duración estimada:** 25 minutos
- **Tipo:** Programación
- **Modalidad:** Individual
- **Dificultad:** Básica
- **Prerequisitos:** Python 3.9+, cuenta en OpenAI con API key o **cuenta gratuita en OpenRouter**

### Contexto

El acceso programático a los LLMs es el primer paso para integrar inteligencia artificial en aplicaciones reales. En este ejercicio realizarás tu primera llamada a la API de un LLM y aprenderás a interpretar la respuesta.

### Objetivo de Aprendizaje

- Configurar el entorno de desarrollo para trabajar con APIs de LLMs
- Realizar una llamada básica a la API de OpenAI
- Interpretar los metadatos de la respuesta (tokens, modelo)
- Comprender el efecto del parámetro `temperature`

### Enunciado

#### Paso 1: Configuración del entorno (5 min)

Instala las dependencias necesarias:

```
pip install openai python-dotenv
```

Crea un archivo `.env` en la raíz de tu proyecto con tu clave:

#### Opción A - OpenAI directo (de pago):

```
OPENAI_API_KEY=sk-tu-clave-aqui
```

#### Opción B - OpenRouter (gratuito):

```
OPENROUTER_API_KEY=sk-or-tu-clave-aqui
```

**Importante:** Nunca subas tu archivo `.env` a un repositorio. Asegúrate de que `.env` esté en tu `.gitignore`.

## Paso 2: Primera llamada a la API (10 min)

Completa el siguiente código rellenando las partes marcadas con `# TODO`:

```
import os
from openai import OpenAI
from dotenv import load_dotenv

# Cargar variables de entorno
load_dotenv()

# TODO: Crear el cliente
# Opción A - OpenAI directo:
# client = OpenAI()
# Opción B - OpenRouter (gratis):
# client = OpenAI(
#     base_url="https://openrouter.ai/api/v1",
#     api_key=os.getenv("OPENROUTER_API_KEY"),
# )
client = ____

# TODO: Realizar la llamada a la API
# Opción A - OpenAI: model="gpt-4o-mini"
# Opción B - OpenRouter: model="google/gemini-2.0-flash-exp:free"
# Envía un mensaje de usuario: "¿Qué es el machine learning? Responde en 3 oraciones."
response = client.chat.completions.create(
    model=____,
    messages=[
        {"role": ____, "content": ____}
    ],
    temperature=0.7
)

# TODO: Extraer e imprimir los siguientes datos:
# 1. El texto de la respuesta
print("Respuesta:", ____)

# 2. El modelo utilizado
print("Modelo:", ____)

# 3. Tokens del prompt
print("Prompt tokens:", ____)

# 4. Tokens de la respuesta
print("Completion tokens:", ____)
```

```
# 5. Total de tokens
print("Total tokens:", ____)
```

### Paso 3: Experimentar con temperature (10 min)

Ejecuta la misma llamada 3 veces con cada uno de estos valores de **temperature**:

| Ejecución | temperature | Observaciones |
|-----------|-------------|---------------|
| A         | 0           |               |
| B         | 0.7         |               |
| C         | 1.5         |               |

Para cada ejecución, documenta:

- ¿La respuesta es idéntica o diferente entre ejecuciones con el mismo temperature?
- ¿Cómo cambia el estilo y la creatividad de la respuesta?
- ¿Qué valor usarías para un asistente de atención al cliente? ¿Y para un generador de poesía?

### Preguntas de Reflexión

1. ¿Por qué es importante monitorear el consumo de tokens?
2. ¿Qué sucede si envías un prompt muy largo? ¿Cómo afecta a los tokens y al costo?
3. ¿Cuál es la diferencia entre **temperature=0** y **temperature=1.5**?

---

## Ejercicio 2: Comparativa de APIs

### Metadata

- **Duración estimada:** 30 minutos
- **Tipo:** Experimentación/Programación
- **Modalidad:** Individual
- **Dificultad:** Intermedia
- **Prerequisitos:** Claves de API de OpenAI, Google Gemini y Anthropic Claude — o solo una cuenta de OpenRouter (gratuita)

### Contexto

En el ecosistema actual existen múltiples proveedores de LLMs. Cada uno tiene sus fortalezas y debilidades. Compararlos de forma objetiva es esencial para tomar decisiones informadas en proyectos reales.

### Objetivo de Aprendizaje

- Interactuar con las APIs de los tres principales proveedores de LLMs
- Medir y comparar tiempos de respuesta y consumo de tokens
- Evaluar cualitativamente las respuestas de cada modelo

## Enunciado

### Paso 1: Configuración (5 min)

#### Opción A - APIs directas (de pago):

Instala las bibliotecas necesarias:

```
pip install openai google-generativeai anthropic python-dotenv
```

Añade a tu archivo `.env`:

```
OPENAI_API_KEY=sk-tu-clave-openai  
GOOGLE_API_KEY=tu-clave-gemini  
ANTHROPIC_API_KEY=sk-ant-tu-clave-anthropic
```

#### Opción B - OpenRouter (gratis):

```
pip install openai python-dotenv
```

```
OPENROUTER_API_KEY=sk-or-tu-clave-aqui
```

Con OpenRouter puedes comparar modelos de distintos proveedores usando una sola API key y modelos gratuitos. Ve directamente al **Paso 2b** más abajo.

### Paso 2: Código para cada API (15 min)

Usa el siguiente prompt para los tres proveedores:

```
Explica qué es la recursividad en programación. Incluye un ejemplo en Python.
```

Completa los esqueletos de código:

#### OpenAI:

```
import os  
import time  
from openai import OpenAI  
from dotenv import load_dotenv
```

```

load_dotenv()
client = OpenAI()

prompt = "Explica qué es la recursividad en programación. Incluye un
ejemplo en Python."

start = time.time()
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.7
)
elapsed = time.time() - start

print("=== OpenAI ===")
print(response.choices[0].message.content)
print(f"\nTiempo: {elapsed:.2f}s")
print(f"Tokens: {response.usage.total_tokens}")

```

### Google Gemini:

```

import os
import time
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()
genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))

model = genai.GenerativeModel("gemini-1.5-flash")

prompt = "Explica qué es la recursividad en programación. Incluye un
ejemplo en Python."

start = time.time()
response = model.generate_content(prompt)
elapsed = time.time() - start

print("=== Google Gemini ===")
print(response.text)
print(f"\nTiempo: {elapsed:.2f}s")
# Nota: Gemini reporta tokens de forma diferente
print(f"Tokens (prompt): {response.usage_metadata.prompt_token_count}")
print(f"Tokens (respuesta):
{response.usage_metadata.candidates_token_count}")

```

### Anthropic Claude:

```
import os
import time
import anthropic
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic()

prompt = "Explica qué es la recursividad en programación. Incluye un ejemplo en Python."

start = time.time()
response = client.messages.create(
    model="claude-3-5-haiku-latest",
    max_tokens=1024,
    messages=[{"role": "user", "content": prompt}]
)
elapsed = time.time() - start

print("=== Anthropic Claude ===")
print(response.content[0].text)
print(f"\nTiempo: {elapsed:.2f}s")
print(f"Tokens (entrada): {response.usage.input_tokens}")
print(f"Tokens (salida): {response.usage.output_tokens}")
```

## Paso 2b: Alternativa con OpenRouter (15 min)

Si usas OpenRouter, puedes comparar modelos de distintos proveedores con una sola configuración:

```
import os
import time
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPENROUTER_API_KEY"),
)

prompt = "Explica qué es la recursividad en programación. Incluye un ejemplo en Python."

# Modelos gratuitos de distintos proveedores disponibles en OpenRouter
# Consulta modelos actualizados en: https://openrouter.ai/models?q=free
modelos = {
    "Google Gemini": "google/gemini-2.0-flash-exp:free",
    "DeepSeek R1": "deepseek/deepseek-r1-0528:free",
```

```

    "Meta Llama 4": "meta-llama/llama-4-scout:free",
}

for nombre, modelo in modelos.items():
    print(f"\n{'='*40}")
    print(f"Modelo: {nombre} ({modelo})")
    print(f"{'='*40}")

    start = time.time()
    response = client.chat.completions.create(
        model=modelo,
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7
    )
    elapsed = time.time() - start

    print(response.choices[0].message.content)
    print(f"\nTiempo: {elapsed:.2f}s")
    if response.usage:
        print(f"Tokens: {response.usage.total_tokens}")

```

### Paso 3: Comparación (10 min)

Completa la siguiente tabla con los resultados:

| Métrica                            | OpenAI | Gemini | Claude |
|------------------------------------|--------|--------|--------|
| Tokens usados (total)              |        |        |        |
| Tiempo de respuesta (s)            |        |        |        |
| Longitud de respuesta (caracteres) |        |        |        |
| Calidad de la explicación (1-10)   |        |        |        |
| Calidad del código Python (1-10)   |        |        |        |
| Calidad subjetiva general (1-10)   |        |        |        |

### Preguntas de Reflexión

1. ¿Cuál de los tres modelos dio la mejor respuesta? ¿Por qué?
2. ¿Cuál fue el más rápido? ¿Crees que la velocidad importa en todos los casos de uso?
3. ¿En qué escenarios elegirías cada proveedor?
4. ¿Notas diferencias en cómo cada modelo estructura su respuesta?

## Ejercicio 3: Chatbot con Memoria

### Metadata

- **Duración estimada:** 35 minutos
- **Tipo:** Programación



- **Modalidad:** Individual
- **Dificultad:** Intermedia
- **Prerequisitos:** Ejercicio 1 completado, comprensión de roles en la API

## Contexto

Los chatbots más útiles son aquellos que recuerdan el contexto de la conversación. Las APIs de LLMs no mantienen estado entre llamadas, por lo que debemos gestionar el historial de mensajes manualmente.

## Objetivo de Aprendizaje

- Implementar gestión de historial de conversación
- Entender cómo funciona la ventana de contexto
- Aplicar límites de tokens mediante recorte de historial
- Usar el rol `system` para definir la personalidad del chatbot

## Enunciado

### Paso 1: Estructura base (10 min)

Completa el siguiente esqueleto implementando la lógica del chatbot:

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

# Opción A - OpenAI directo:
client = OpenAI()
MODEL = "gpt-4o-mini"

# Opción B - OpenRouter (gratuito): descomenta las siguientes líneas y
# comenta las anteriores
# client = OpenAI(
#     base_url="https://openrouter.ai/api/v1",
#     api_key=os.getenv("OPENROUTER_API_KEY"),
# )
# MODEL = "google/gemini-2.0-flash-exp:free"

# TODO: Define el system prompt para un tutor de Python amigable
# Debe presentarse, ser paciente y dar ejemplos claros
SYSTEM_PROMPT = ____

# Límite máximo de mensajes en el historial (sin contar el system prompt)
MAX_MESSAGES = 10

def create_initial_messages():
    """Crea la lista inicial de mensajes con el system prompt."""
    # TODO: Retorna una lista con el mensaje de sistema
    return ____
```

```
def trim_history(messages):
    """
    Recorta el historial si excede MAX_MESSAGES.
    Mantiene siempre el system prompt (primer mensaje)
    y los últimos MAX_MESSAGES mensajes.
    """
    # TODO: Implementa la lógica de recorte
    # Pista: messages[0] es el system prompt, el resto es la conversación
    if len(messages) - 1 > MAX_MESSAGES:
        _____
    return messages

def get_response(messages):
    """Envía los mensajes a la API y retorna la respuesta."""
    # TODO: Realiza la llamada a la API usando MODEL y retorna el objeto
    response
    response = _____
    return response

def chat():
    """Bucle principal del chatbot."""
    messages = create_initial_messages()
    print("=" * 50)
    print("  Tutor de Python - Escribe 'salir' para terminar")
    print("=" * 50)
    print()

    while True:
        user_input = input("Tú: ").strip()

        if user_input.lower() == "salir":
            print("\n¡Hasta pronto! Sigue practicando Python.")
            break

        if not user_input:
            continue

        # TODO: Añadir el mensaje del usuario al historial
        _____

        # TODO: Recortar historial si es necesario
        messages = _____

        # TODO: Obtener respuesta de la API
        response = _____

        # TODO: Extraer el texto de la respuesta
        assistant_message = _____

        # TODO: Añadir la respuesta del asistente al historial
        _____

        # Mostrar respuesta y estadísticas
```

```
print(f"\nTutor: {assistant_message}")
print(f"  [Tokens - Prompt: {response.usage.prompt_tokens}, "
      f"  Respuesta: {response.usage.completion_tokens}, "
      f"  Total: {response.usage.total_tokens}]\n")
print(f"  [Mensajes en historial: {len(messages) - 1}]\n")
print()

if __name__ == "__main__":
    chat()
```

## Paso 2: Prueba de memoria (10 min)

Una vez implementado, prueba la siguiente secuencia de conversación:

1. Escribe: "¿Qué son las variables en Python?"
2. Escribe: "Dame un ejemplo de lo anterior"
3. Escribe: "Ahora muéstrame cómo usar listas"
4. Escribe: "¿Cuál es la diferencia entre lo primero que me explicaste y esto?"

Verifica que el chatbot:

- Recuerda el contexto de mensajes anteriores
- En la pregunta 2, sabe que "lo anterior" se refiere a variables
- En la pregunta 4, puede comparar variables con listas

## Paso 3: Probar el límite de historial (10 min)

Cambia `MAX_MESSAGES = 4` y mantén una conversación larga. Observa:

- ¿En qué momento el chatbot "olvida" los primeros mensajes?
- ¿Cómo afecta esto a la coherencia de la conversación?
- ¿Cómo cambia el consumo de tokens cuando el historial se recorta?

## Paso 4 (Bonus): Resumen de historial (5 min extra)

Si terminas antes, implementa una mejora: cuando el historial se recorte, en lugar de simplemente eliminar los mensajes antiguos, genera un resumen de los mensajes eliminados y añádelo como un mensaje de sistema adicional.

```
def summarize_and_trim(messages):
    """
    En lugar de simplemente recortar, resume los mensajes que se van a
    eliminar
    y añade el resumen como contexto.
    """
    if len(messages) - 1 > MAX_MESSAGES:
        # Mensajes que se van a eliminar
        old_messages = messages[1:-MAX_MESSAGES]

        # Generar resumen de los mensajes antiguos
```

```
summary_prompt = [
    {"role": "system", "content": "Resume brevemente los siguientes intercambios en 2-3 oraciones:"},
    *old_messages
]
# TODO: Llamar a la API para generar el resumen
# TODO: Insertar el resumen como segundo mensaje (después del system prompt)
# TODO: Mantener solo los últimos MAX_MESSAGES mensajes de conversación

return messages
```

## Preguntas de Reflexión

1. ¿Por qué las APIs de LLMs no mantienen el estado entre llamadas?
2. ¿Qué ventajas y desventajas tiene limitar el historial a 10 mensajes?
3. ¿Cómo resolverías el problema de contexto en conversaciones muy largas en un producto real?

---

## Ejercicio 4: Extracción Estructurada

### Metadata

- **Duración estimada:** 25 minutos
- **Tipo:** Programación
- **Modalidad:** Individual
- **Dificultad:** Avanzada
- **Prerequisitos:** Ejercicios 1 y 2 completados, conocimiento básico de JSON

### Contexto

Una de las aplicaciones más valiosas de los LLMs en entornos empresariales es la extracción de datos estructurados a partir de texto libre. Esto permite automatizar procesos que antes requerían lectura manual.

### Objetivo de Aprendizaje

- Diseñar prompts que produzcan salidas en formato JSON válido
- Implementar validación y manejo de errores para respuestas de LLMs
- Construir lógica de reintentos cuando la respuesta no cumple el formato esperado

### Enunciado

#### Paso 1: Definir el sistema de extracción (5 min)

Usa el siguiente system prompt como base:

```
SYSTEM_PROMPT = """Eres un sistema de extracción de información. Tu tarea es extraer datos estructurados de textos no estructurados y devolver ÚNICAMENTE un JSON válido.
```

Reglas:

- Responde SÓLO con el JSON, sin texto adicional, sin bloques de código markdown.
  - Si un campo no se encuentra en el texto, usa el valor "No especificado".
  - Los valores numéricos deben ser números, no strings.
  - Las fechas deben estar en formato YYYY-MM-DD cuando sea posible.
- ```
"""
```

## Paso 2: Textos de entrada y esquemas esperados (5 min)

### Texto 1 - Oferta de empleo:

```
texto_empleo = """
¡Únete a nuestro equipo! Buscamos Desarrollador Senior Python para nuestra oficina en Madrid. Ofrecemos salario de 45.000-55.000€ brutos anuales, teletrabajo 3 días por semana y seguro médico privado. Requisitos: 5 años de experiencia, conocimientos en Django y PostgreSQL. Incorporación inmediata.
Enviar CV a empleo@techcorp.es antes del 15 de marzo de 2025.
"""
```

### Esquema esperado:

```
{
  "puesto": "string",
  "empresa": "string",
  "ubicacion": "string",
  "salario_min": "number",
  "salario_max": "number",
  "modalidad": "string",
  "requisitos": ["string"],
  "beneficios": ["string"],
  "contacto": "string",
  "fecha_limite": "string (YYYY-MM-DD)"
}
```

### Texto 2 - Reseña de producto:

```
texto_resena = """
Compré el portátil UltraBook X15 hace 2 semanas. La pantalla de 15 pulgadas es espectacular y la batería dura unas 10 horas reales. Sin embargo, el
```

```
teclado es un poco incómodo para escribir largo rato y se calienta bastante con tareas pesadas. Por el precio de 1.299€ creo que está bien, pero no es perfecto. Le doy un 7 de 10. Lo compré en Amazon el 20 de enero de 2025.
"""
```

Esquema esperado:

```
{
  "producto": "string",
  "puntuacion": "number",
  "puntuacion_maxima": "number",
  "precio": "number",
  "aspectos_positivos": ["string"],
  "aspectos_negativos": ["string"],
  "fecha_compra": "string (YYYY-MM-DD)",
  "tienda": "string",
  "recomendacion_general": "string (positiva/neutra/negativa)"
}
```

**Texto 3 - Noticia:**

```
texto_noticia = """
La empresa española de inteligencia artificial, NovaTech, anunció hoy una ronda de financiación Serie B por valor de 30 millones de euros, liderada por el fondo Sequoia Capital con participación de Telefónica Ventures. La compañía, fundada en 2021 por María García y Carlos López, planea usar los fondos para expandirse a Latinoamérica y contratar a 50 ingenieros antes de fin de año. NovaTech ha desarrollado un modelo de lenguaje especializado en el sector legal.
"""
```

Esquema esperado:

```
{
  "empresa": "string",
  "tipo_evento": "string",
  "monto": "number",
  "moneda": "string",
  "inversores": ["string"],
  "fundadores": ["string"],
  "año_fundacion": "number",
  "sector": "string",
  "planes": ["string"]
}
```

**Paso 3: Implementar la extracción con validación (10 min)**

```

import os
import json
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

# Opción A - OpenAI directo:
client = OpenAI()
MODEL = "gpt-4o-mini"

# Opción B - OpenRouter (gratuito): descomenta las siguientes líneas y
# comenta las anteriores
# client = OpenAI(
#     base_url="https://openrouter.ai/api/v1",
#     api_key=os.getenv("OPENROUTER_API_KEY"),
# )
# MODEL = "google/gemini-2.0-flash-exp:free"

SYSTEM_PROMPT = """Eres un sistema de extracción de información. Tu tarea
es extraer datos
estructurados de textos no estructurados y devolver ÚNICAMENTE un JSON
válido.

Reglas:
- Responde SOLO con el JSON, sin texto adicional, sin bloques de código
markdown.
- Si un campo no se encuentra en el texto, usa el valor "No especificado".
- Los valores numéricos deben ser números, no strings.
- Las fechas deben estar en formato YYYY-MM-DD cuando sea posible.
"""

def extract_json(text, schema_description, max_retries=3):
    """
    Extrae datos estructurados de un texto libre.

    Args:
        text: Texto del cual extraer información.
        schema_description: Descripción del esquema JSON esperado.
        max_retries: Número máximo de reintentos si el JSON es inválido.

    Returns:
        dict: Datos extraídos como diccionario Python.
    """
    user_prompt = f"""Extrae la información del siguiente texto y devuelve
un JSON
con este esquema:

{schema_description}

Texto:
\"\"\"

```

```

{text}
\"\"\"
"""

for attempt in range(max_retries):
    try:
        response = client.chat.completions.create(
            model=MODEL,
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_prompt}
            ],
            temperature=0
        )

        content = response.choices[0].message.content.strip()

        # Limpiar posibles bloques de código markdown
        if content.startswith("```"):
            content = content.split("\n", 1)[1]
            content = content.rsplit("```", 1)[0]

        result = json.loads(content)
        print(f" [Extracción exitosa en intento {attempt + 1}]")
        return result

    except json.JSONDecodeError as e:
        print(f" [Intento {attempt + 1}/{max_retries}] JSON inválido:
{e}")

        if attempt < max_retries - 1:
            print(" Reintentando...")
        else:
            print(" Se agotaron los reintentos.")
            return None

# Ejecutar extracciones
textos = {
    "Oferta de empleo": (texto_empleo, "puesto, empresa, ubicacion,
salario_min, salario_max, modalidad, requisitos (lista), beneficios
(lista), contacto, fecha_limite"),
    "Reseña de producto": (texto_resena, "producto, puntuacion,
puntuacion_maxima, precio, aspectos_positivos (lista), aspectos_negativos
(lista), fecha_compra, tienda, recomendacion_general"),
    "Noticia": (texto_noticia, "empresa, tipo_evento, monto, moneda,
inversores (lista), fundadores (lista), año_fundacion, sector, planes
(lista)")
}

for nombre, (texto, esquema) in textos.items():
    print(f"\n{'='*50}")
    print(f"Procesando: {nombre}")
    print(f"{'='*50}")
    resultado = extract_json(texto, esquema)

```



```
if resultado:
    print(json.dumps(resultado, indent=2, ensure_ascii=False))
```

#### Paso 4: Análisis (5 min)

Responde:

1. ¿En cuántos intentos logró generar JSON válido para cada texto?
2. ¿Hubo campos con valor "No especificado"? ¿Era correcto?
3. ¿Los valores numéricos fueron números o strings?
4. ¿Qué pasaría si el texto de entrada estuviera en otro idioma?

---

## Ejercicio 5: Introducción a LangChain

### Metadata

- **Duración estimada:** 30 minutos
- **Tipo:** Programación
- **Modalidad:** Individual
- **Dificultad:** Intermedia
- **Prerequisitos:** Ejercicio 4 completado, familiaridad con el patrón pipe en Python

### Contexto

LangChain es un framework que simplifica la construcción de aplicaciones con LLMs. Permite encadenar componentes (modelos, prompts, parsers) de forma declarativa, reduciendo el código repetitivo y facilitando el mantenimiento.

### Objetivo de Aprendizaje

- Instalar y configurar LangChain con OpenAI
- Crear cadenas (chains) usando el operador pipe (|)
- Comparar la complejidad del código nativo vs. LangChain
- Entender cuándo usar un framework vs. acceso directo a la API

### Enunciado

#### Paso 1: Instalación y configuración (5 min)

```
pip install langchain langchain-openai
```

Verifica la instalación:

```
import langchain
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser

print(f"LangChain versión: {langchain.__version__}")
```

## Paso 2: Tu primera chain (10 min)

Replica la extracción estructurada del Ejercicio 4 usando LangChain:

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

# 1. Crear el modelo
# Opción A - OpenAI directo:
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# Opción B - OpenRouter (gratuito): descomenta y comenta la línea anterior
# model = ChatOpenAI(
#     model="google/gemini-2.0-flash-exp:free",
#     temperature=0,
#     openai_api_key=os.getenv("OPENROUTER_API_KEY"),
#     openai_api_base="https://openrouter.ai/api/v1",
# )

# 2. Crear el template del prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", """Eres un sistema de extracción de información. Tu tarea es
    extraer datos
    estructurados de textos no estructurados y devolver ÚNICAMENTE un JSON
    válido.

    Reglas:
    - Responde SOLO con el JSON, sin texto adicional, sin bloques de código
    markdown.
    - Si un campo no se encuentra en el texto, usa el valor "No especificado".
    - Los valores numéricos deben ser números, no strings.
    - Las fechas deben estar en formato YYYY-MM-DD cuando sea posible."""),
    ("user", """Extrae la información del siguiente texto y devuelve un
    JSON
    con este esquema: {schema}

    Texto:
    \"\"\"
    {text}
    \"\"\"
    """)
])
```

```
# 3. Crear el parser de salida
output_parser = StrOutputParser()

# 4. Crear la chain con el operador pipe
chain = prompt | model | output_parser

# 5. Invocar la chain
result = chain.invoke({
    "text": texto_empleo,
    "schema": "puesto, empresa, ubicacion, salario_min, salario_max,
modalidad, requisitos (lista), beneficios (lista), contacto, fecha_limite"
})

print(result)
```

### Paso 3: Procesar los tres textos (10 min)

Usa la misma chain para procesar los tres textos del Ejercicio 4:

```
import json

textos = {
    "Oferta de empleo": {
        "text": texto_empleo,
        "schema": "puesto, empresa, ubicacion, salario_min, salario_max,
modalidad, requisitos (lista), beneficios (lista), contacto, fecha_limite"
    },
    "Reseña de producto": {
        "text": texto_resena,
        "schema": "producto, puntuacion, puntuacion_maxima, precio,
aspectos_positivos (lista), aspectos_negativos (lista), fecha_compra,
tienda, recomendacion_general"
    },
    "Noticia": {
        "text": texto_noticia,
        "schema": "empresa, tipo_evento, monto, moneda, inversores (lista),
fundadores (lista), año_fundacion, sector, planes (lista)"
    }
}

for nombre, inputs in textos.items():
    print(f"\n{'='*50}")
    print(f"Procesando: {nombre}")
    print(f"{'='*50}")
    result = chain.invoke(inputs)
    try:
        parsed = json.loads(result)
        print(json.dumps(parsed, indent=2, ensure_ascii=False))
```

```
except json.JSONDecodeError:
    print(f"Error al parsear JSON: {result}")
```

#### Paso 4: Comparación (5 min)

Completa la siguiente tabla:

| Aspecto                   | API Nativa (Ej. 4) | LangChain (Ej. 5) |
|---------------------------|--------------------|-------------------|
| Líneas de código (aprox.) |                    |                   |
| Facilidad de lectura      |                    |                   |
| Gestión de reintentos     |                    |                   |
| Cambiar de modelo         |                    |                   |
| Curva de aprendizaje      |                    |                   |

#### Preguntas de Reflexión

1. ¿Cuántas líneas de código te ahorraste con LangChain?
2. ¿Qué beneficios ves en el patrón `prompt | model | parser`?
3. ¿En qué situaciones NO usarías LangChain y preferirías la API nativa?
4. ¿Cómo cambiarías la chain para usar Claude en vez de OpenAI?

## Ejercicio Extra: Cliente Multi-Proveedor

### Metadata

- **Duración estimada:** 45 minutos (tarea para casa)
- **Tipo:** Programación/Diseño
- **Modalidad:** Individual
- **Dificultad:** Avanzada
- **Prerequisitos:** Todos los ejercicios anteriores completados

### Enunciado

Construye una clase unificada que abstraiga las diferencias entre los tres proveedores principales de LLMs, ofreciendo una interfaz consistente independientemente del proveedor elegido.

### Requisitos Funcionales

1. **Clase `LLMClient`** con los siguientes métodos:
  - `__init__(self, provider)` - Inicializa el cliente según el proveedor (`"openai"`, `"gemini"`, `"claude"`)
  - `chat(self, messages, **kwargs)` - Envía mensajes y retorna la respuesta como string
  - `stream(self, messages)` - Envía mensajes y retorna un generador que produce tokens uno a uno

## 2. Formato de mensajes unificado:

```
messages = [  
    {"role": "system", "content": "Eres un asistente útil."},  
    {"role": "user", "content": "Hola, ¿cómo estás?"}  
]
```

## 3. Manejo de diferencias entre APIs:

- Gemini no tiene rol **system** nativo: concatenar al primer mensaje de usuario
- Claude usa **system** como parámetro separado, no dentro de **messages**
- Los nombres de modelos por defecto deben ser configurables

## Estructura Sugerida

```
import os  
from dotenv import load_dotenv  
  
load_dotenv()  
  
class LLMClient:  
    """Cliente unificado para múltiples proveedores de LLMs."""  
  
    DEFAULT_MODELS = {  
        "openai": "gpt-4o-mini",  
        "gemini": "gemini-1.5-flash",  
        "claude": "claude-3-5-haiku-latest",  
        "openrouter": "google/gemini-2.0-flash-exp:free" # Alternativa  
gratuita  
    }  
  
    def __init__(self, provider: str, model: str = None):  
        """  
        Inicializa el cliente.  
  
        Args:  
            provider: Proveedor a usar ("openai", "gemini", "claude",  
"openrouter").  
            model: Modelo específico. Si es None, usa el modelo por  
defecto.  
        """  
        if provider not in self.DEFAULT_MODELS:  
            raise ValueError(f"Proveedor no soportado: {provider}. Usa:  
{list(self.DEFAULT_MODELS.keys())}")  
  
        self.provider = provider  
        self.model = model or self.DEFAULT_MODELS[provider]  
  
        # TODO: Inicializar el cliente del proveedor correspondiente  
        # Pista: usa if/elif para cada proveedor
```

```

        # Para OpenRouter, usa
        OpenAI(base_url="https://openrouter.ai/api/v1", api_key=...)

    def _adapt_messages(self, messages):
        """
        Adapta los mensajes al formato que espera cada proveedor.

        Returns:
            dict con las claves necesarias para cada API.
        """
        # TODO: Implementar la adaptación de mensajes
        # - OpenAI: messages tal cual
        # - Gemini: convertir a formato de Gemini, system va aparte
        # - Claude: separar system de messages
        pass

    def chat(self, messages, **kwargs):
        """
        Envía mensajes al LLM y retorna la respuesta.

        Args:
            messages: Lista de mensajes en formato unificado.
            **kwargs: Parámetros adicionales (temperature, max_tokens,
etc.)

        Returns:
            str: Texto de la respuesta.
        """
        # TODO: Implementar para cada proveedor
        pass

    def stream(self, messages, **kwargs):
        """
        Envía mensajes al LLM y retorna un generador de tokens.

        Yields:
            str: Cada token/fragmento de la respuesta.
        """
        # TODO: Implementar streaming para cada proveedor
        pass

```

## Código de Prueba

Usa el siguiente código para validar tu implementación:

```

# Probar los tres proveedores con el mismo prompt
messages = [
    {"role": "system", "content": "Eres un asistente conciso. Responde en máximo 2 oraciones."},
    {"role": "user", "content": "¿Qué es Python?"}
]

```

```
for provider in ["openai", "gemini", "claude"]:
    print(f"\n{'='*40}")
    print(f"Proveedor: {provider}")
    print(f"{'='*40}")

    try:
        client = LLMClient(provider)
        response = client.chat(messages, temperature=0.7)
        print(f"Respuesta: {response}")
    except Exception as e:
        print(f"Error: {e}")

# Probar streaming con OpenAI
print(f"\n{'='*40}")
print("Streaming con OpenAI")
print(f"{'='*40}")

client = LLMClient("openai")
for token in client.stream(messages):
    print(token, end="", flush=True)
print()
```

## Entregable

- Archivo `llm_client.py` con la clase completa
- Archivo `test_client.py` con las pruebas
- Breve documentación sobre las decisiones de diseño tomadas

---

## Soluciones de Referencia

- ▶ Ver solución Ejercicio 1 - Primera Llamada a la API
- ▶ Ver solución Ejercicio 3 - Chatbot con Memoria
- ▶ Ver solución Ejercicio 4 - Extracción Estructurada